

# HIGH PERFORMANCE COMPUTING

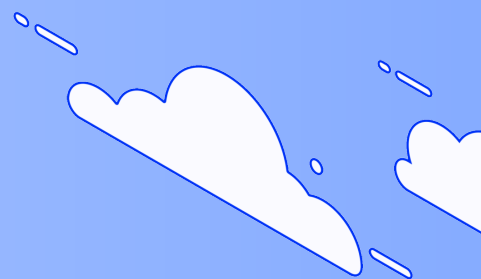
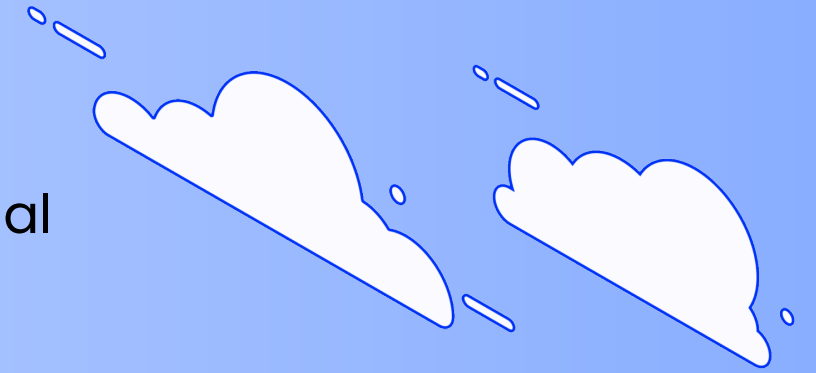
## UNIT 4

Analytical Modeling of Parallel  
Programs



# SOURCES OF OVERHEAD IN PARALLEL PROGRAMS

Overhead refers to the extra time and resources required to manage parallel execution apart from actual computation.



**Communication Overhead** – Time spent in data transfer between processors.

**Synchronization Overhead** – Waiting time during coordination among processes.

**Load Imbalance** – Unequal distribution of work causes some processors to remain idle.

**Thread/Process Management** – Time required for creation, scheduling, and switching.

**Memory Access Overhead** – Delays due to shared memory access and cache coherence.

# PERFORMANCE MEASURES AND ANALYSIS

## 1. Amdahl's Law

Amdahl's Law states that the speedup of a parallel program is limited by the portion of the program that must execute sequentially.

Even if many processors are used, the sequential part becomes the bottleneck.

Formula

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Where:

- S = Overall speedup
- P = Parallel portion of the program
- (1 - P) = Sequential portion
- N = Number of processors

Example

Suppose:

- 80% of a program can be parallelized
- 20% is sequential
- 4 processors are used

Then:

$$S = \frac{1}{0.2 + \frac{0.8}{4}} = 2.5$$

So the maximum speedup is 2.5 times.

# PERFORMANCE MEASURES AND ANALYSIS

## 2. Gustafson’s Law

### Definition

Gustafson’s Law states that by increasing the problem size with the number of processors, better speedup can be achieved. It assumes that larger problems can efficiently utilize more processors.

### Formula

$$S=N-(1-P)(N-1)$$

### Where:

- S = Speedup
- N = Number of processors
- P = Parallel portion of the program

### Example

#### Suppose:

- Parallel portion = 90%
- Processors = 8

#### Then:

$$s=8-(0.1)(7)=7.3$$

So the speedup is approximately 7.3 times.

| Amdahl’s Law                   | Gustafson’s Law                  |
|--------------------------------|----------------------------------|
| Assumes fixed problem size     | Assumes increasing problem size  |
| Sequential part limits speedup | Larger workloads improve speedup |
| Gives pessimistic results      | Gives optimistic results         |
| Focus on limitations           | Focus on scalability             |



# SPEEDUP FACTOR AND EFFICIENCY IN PARALLEL COMPUTING

## Speedup Factor

- Measures how much faster a parallel program runs compared to a sequential program.

### Formula

$$S = \frac{T_1}{T_p}$$

Where:

- $T_1$  = Sequential execution time
- $T_p$  = Parallel execution time

### Example

If:

- $T_1 = 100s$
- $T_p = 25s$

Then:

$$S = \frac{100}{25} = 4$$

Speedup = 4

## Efficiency

- Measures how effectively processors are utilized.

### Formula

$$E = \frac{S}{P}$$

Where:

- $S$  = Speedup
- $P$  = Number of processors

### Example

If:

- Speedup = 4
- Processors = 5

Then:

$$E = \frac{4}{5} = 0.8 = 80\%$$

Efficiency = 80%

| Speedup                          | Efficiency                       |
|----------------------------------|----------------------------------|
| Measures performance improvement | Measures processor utilization   |
| Ratio of execution times         | Ratio of speedup to processors   |
| Indicates faster execution       | Indicates resource effectiveness |

# COST AND UTILIZATION IN PARALLEL COMPUTING

## 1. Cost

Cost in parallel computing represents the total amount of work done by all processors during execution.

It is used to evaluate whether the parallel algorithm is economical or not.

### Formula

$$\text{Cost} = P \times T_p$$

Where:

- $P$  = Number of processors
- $T_p$  = Parallel execution time

### Example

If:

- Number of processors = 4
- Parallel execution time = 20 sec

Then:

$$\text{Cost} = 4 \times 20 = 80$$

So, total cost = 80 processor-seconds.

## 2. Utilization

Utilization measures how effectively processors are being used during execution.

It indicates the percentage of time processors remain busy.

### Formula

$$\text{Utilization} = \frac{\text{Busy Time}}{\text{Total Time}}$$

### Example

If:

- Processor busy time = 90 sec
- Total available time = 100 sec

Then:

$$\text{Utilization} = \frac{90}{100} = 0.9 = 90\%$$

So, utilization = 90%.

# EXECUTION RATE AND REDUNDANCY IN PARALLEL COMPUTING

## 1. Execution Rate

- Execution Rate represents the amount of work completed per unit time in a computer system.
- It indicates how fast the system executes tasks or instructions.

### Formula

$$\text{Execution Rate} = \frac{\text{Number of Tasks}}{\text{Execution Time}}$$

### Example

If:

- 100 tasks are completed
- Total execution time = 20 sec

Then:

$$\text{Execution Rate} = \frac{100}{20} = 5 \text{ tasks/sec}$$

So, the system executes 5 tasks per second.

## 2. Redundancy

- Redundancy means performing extra operations or using extra hardware/resources to improve reliability and fault tolerance.
- In parallel computing, duplicate processors or computations may be used so that if one component fails, another can continue the work.

### Types of Redundancy

1. Hardware Redundancy
2. Software Redundancy
3. Information Redundancy

### Example

Two processors perform the same task simultaneously.

- If one processor fails,
- The second processor still provides the result.

# THE EFFECT OF GRANULARITY ON PERFORMANCE

- Granularity in parallel computing refers to the amount of computation performed between communication or synchronization operations.
- It affects the overall performance and efficiency of a parallel system.

## TYPES OF GRANULARITY

### 1. Fine-Grained Parallelism

- Tasks are divided into very small parts.
- Frequent communication occurs between processors.

#### Characteristics

- High communication overhead
- Better load balancing
- Suitable for tightly coupled systems

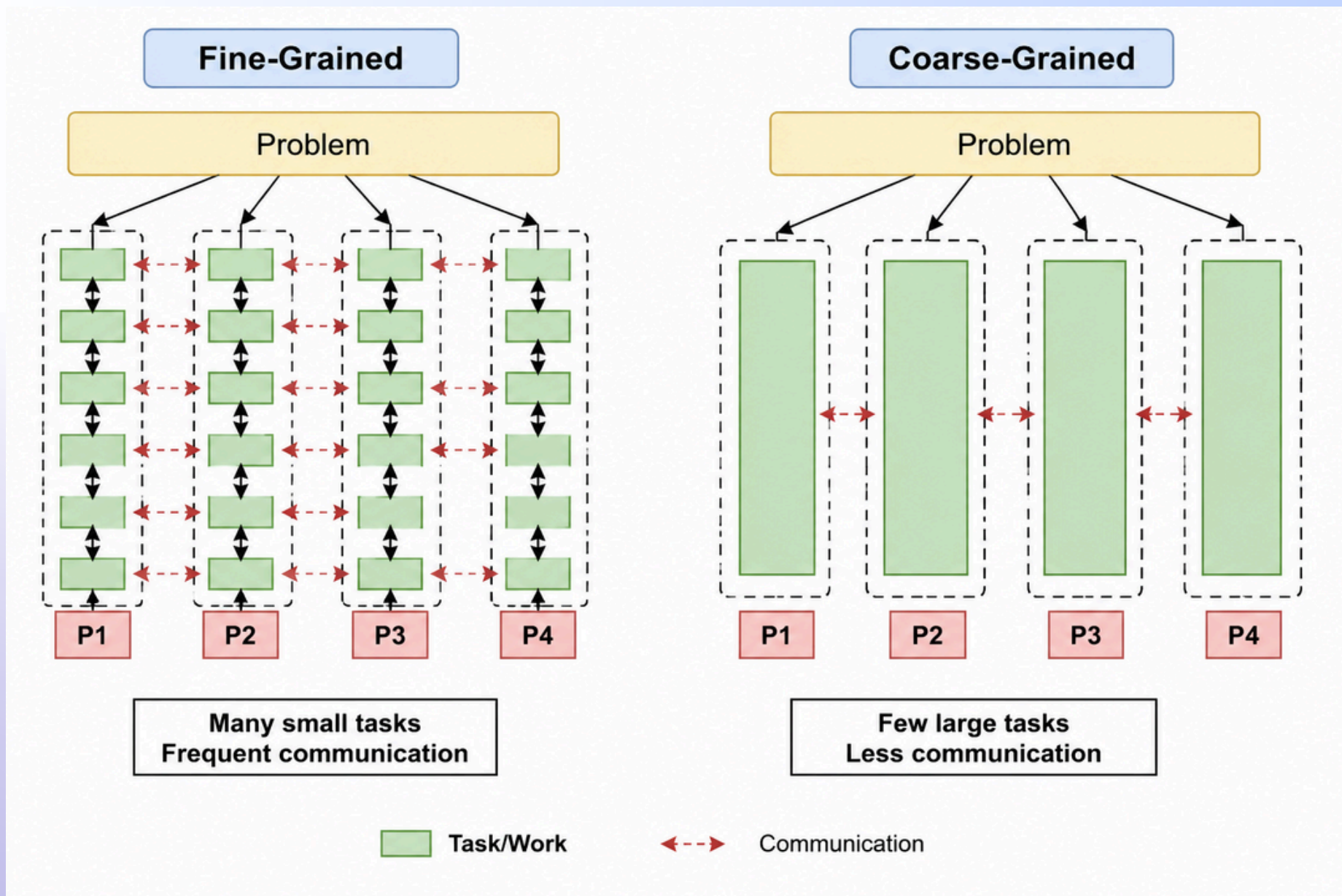
### 2. Coarse-Grained Parallelism

- Tasks are divided into larger parts.
- Less communication between processors.

#### Characteristics

- Lower communication overhead
- Better efficiency
- Less synchronization needed





| Fine-Grained                   | Coarse-Grained              |
|--------------------------------|-----------------------------|
| More communication overhead    | Less communication overhead |
| Better load balancing          | Possible load imbalance     |
| Higher synchronization cost    | Lower synchronization cost  |
| Lower efficiency in some cases | Higher efficiency           |

# SCALABILITY OF PARALLEL SYSTEMS

- Scalability refers to the ability of a parallel system to maintain or improve performance when the number of processors or workload increases.
- A scalable system can efficiently use additional processors to solve larger problems faster.

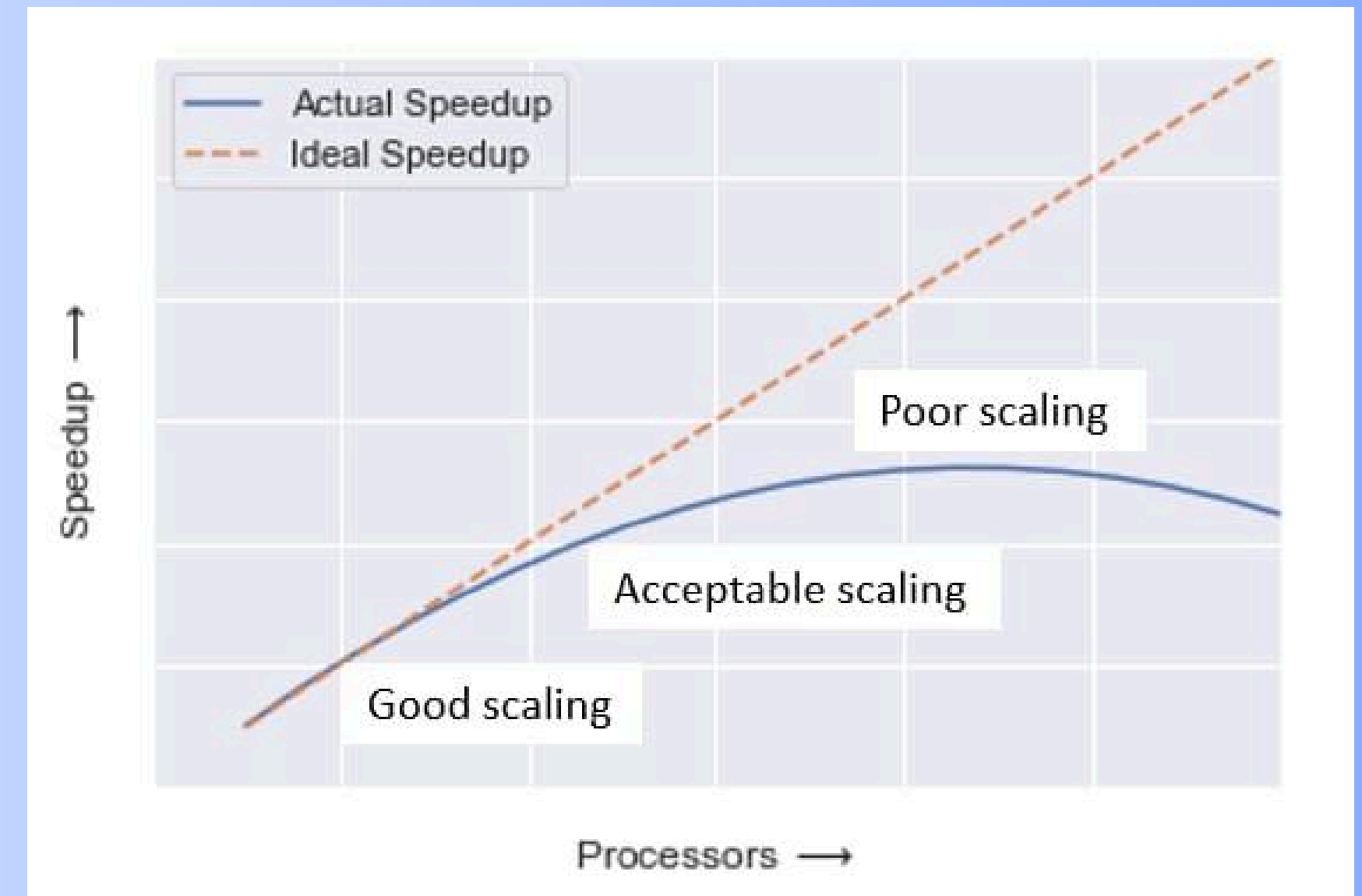
## Types of Scalability

### 1. Strong Scalability

- Problem size remains fixed.
- Performance improves by increasing processors.
- Example
- Same task is completed faster using more processors.

### 2. Weak Scalability

- Problem size increases along with processors.
- Execution time remains nearly constant.
- Example
- More processors handle larger workloads efficiently.





# MINIMUM EXECUTION TIME AND MINIMUM COST

## 1. Minimum Execution Time

- Minimum Execution Time refers to the smallest possible time required to complete a program using parallel processors.
- The main goal is to finish the task as fast as possible by increasing parallelism.

### Formula

$$T_p = \frac{T_1}{P}$$

Where:

- $T_p$  = Parallel execution time
- $T_1$  = Sequential execution time
- $P$  = Number of processors

### Example

If:

- Sequential execution time = 100 sec
- Number of processors = 5

Then:

$$T_p = \frac{100}{5} = 20 \text{ sec}$$

So, minimum execution time = 20 seconds.

## 2. Minimum Cost

- Minimum Cost means achieving the required performance using the least amount of resources.

In parallel computing, cost depends on:

- Number of processors
- Execution time

### Formula

$$\text{Cost} = P \times T_p$$

Where:

- $P$  = Number of processors
- $T_p$  = Parallel execution time

### Example

If:

- Processors = 4
- Parallel execution time = 15 sec

Then:

$$\text{Cost} = 4 \times 15 = 60$$

So, minimum cost = 60 processor-seconds.

# OPTIMAL EXECUTION TIME

- Optimal Execution Time is the best possible execution time achieved in a parallel system with efficient utilization of processors and minimum overhead.
- It means the program executes as fast as possible without wasting resources.

## Formula

$$T_{optimal} = \frac{T_1}{P}$$

Where:

- $T_{optimal}$  = Optimal execution time
- $T_1$  = Sequential execution time
- $P$  = Number of processors

## Example

If:

- Sequential execution time = 120 sec
- Number of processors = 6

Then:

$$T_{optimal} = \frac{120}{6} = 20 \text{ sec}$$

So, optimal execution time = **20 seconds**.

# ASYMPTOTIC ANALYSIS OF PARALLEL PROGRAMS

Asymptotic Analysis is used to study the performance of parallel programs as the input size becomes very large.

It helps measure:

- Execution time
- Speedup
- Scalability
- Efficiency

without depending on specific hardware details.

## Purpose

- Compare parallel algorithms
- Predict performance for large inputs
- Analyze processor utilization
- Estimate overhead and scalability

## Common Asymptotic Notations

### 1. Big O Notation $O$

Represents the upper bound of execution time.

Example

$O(n^2)$

Worst-case complexity.

### 2. Omega Notation $\Omega$

Represents the lower bound of execution time.

Example

$\Omega(n)$

Best-case complexity.

### 3. Theta Notation $\Theta$

Represents the exact bound.

Example

$\Theta(n \log n)$

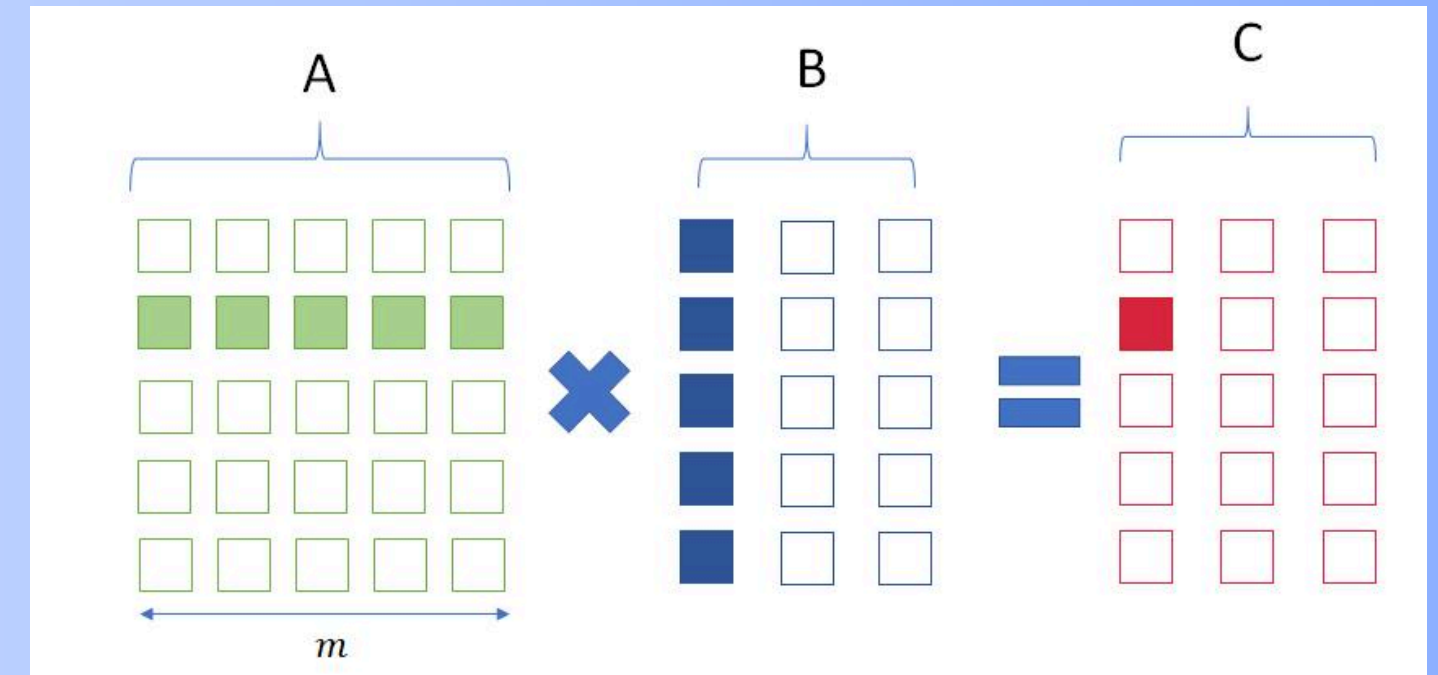
Average or exact complexity.

# MATRIX-VECTOR MULTIPLICATION

Matrix-Vector Multiplication is an operation in which a matrix is multiplied by a vector to produce another vector.

It is widely used in:

- Parallel computing
- Machine learning
- Graphics
- Scientific computing



## Matrix and Vector Representation

Suppose:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

and

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Then the result is:

$$Y = A \times X$$

## Formula

Each element of the output vector is calculated as:

$$y_i = \sum_{j=1}^n a_{ij} x_j$$

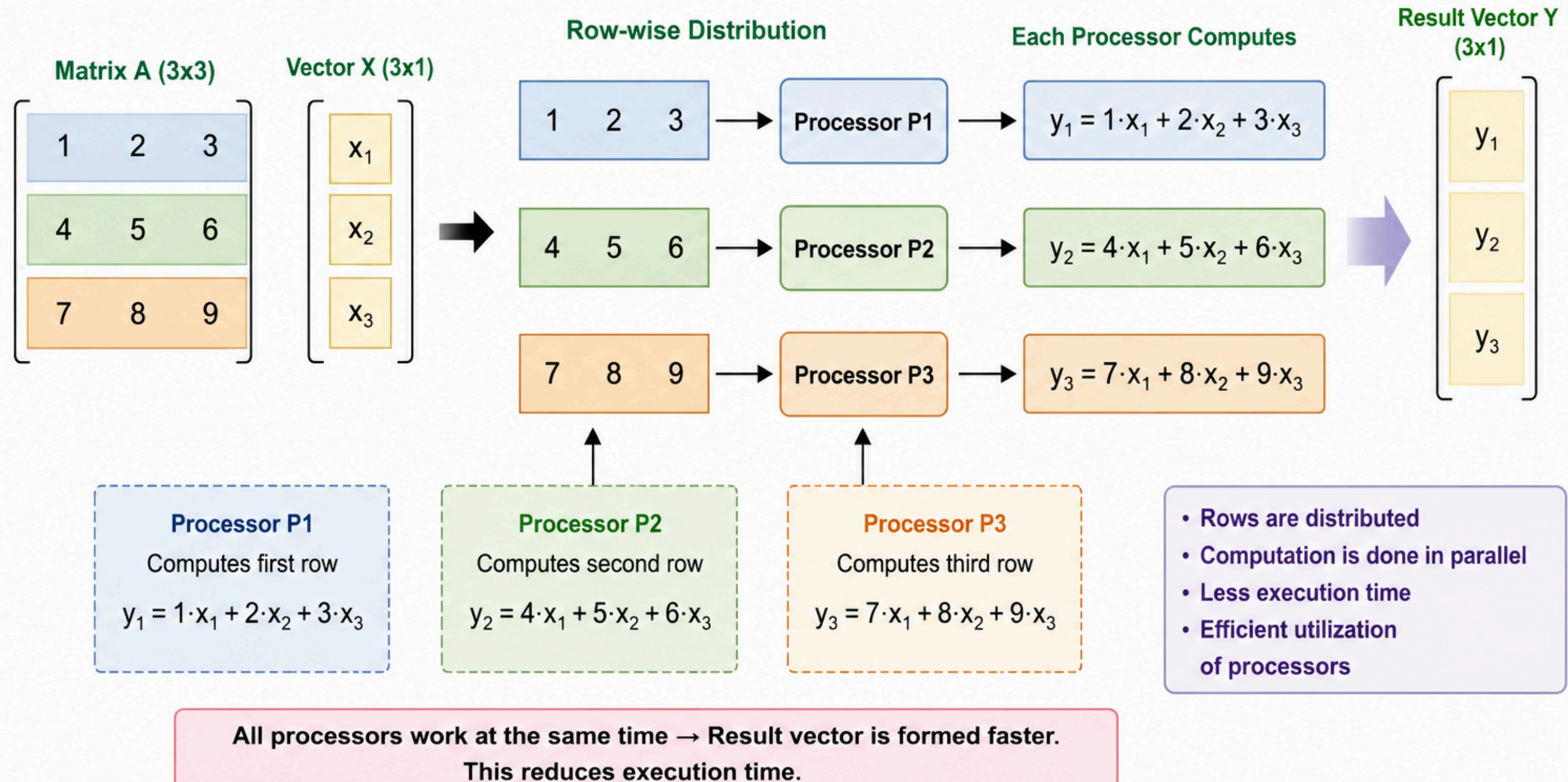
Where:

- $a_{ij}$  = Matrix elements
- $x_j$  = Vector elements
- $y_i$  = Result vector elements



# Parallel Matrix–Vector Multiplication

- Different rows of the matrix are assigned to different processors.
- Each processor computes part of the result vector simultaneously.



# MATRIX-VECTOR MULTIPLICATION USING PARTITIONING

## i) Row-wise 1D Partitioning

### Definition

In Row-wise 1D partitioning, the matrix rows are divided among processors.

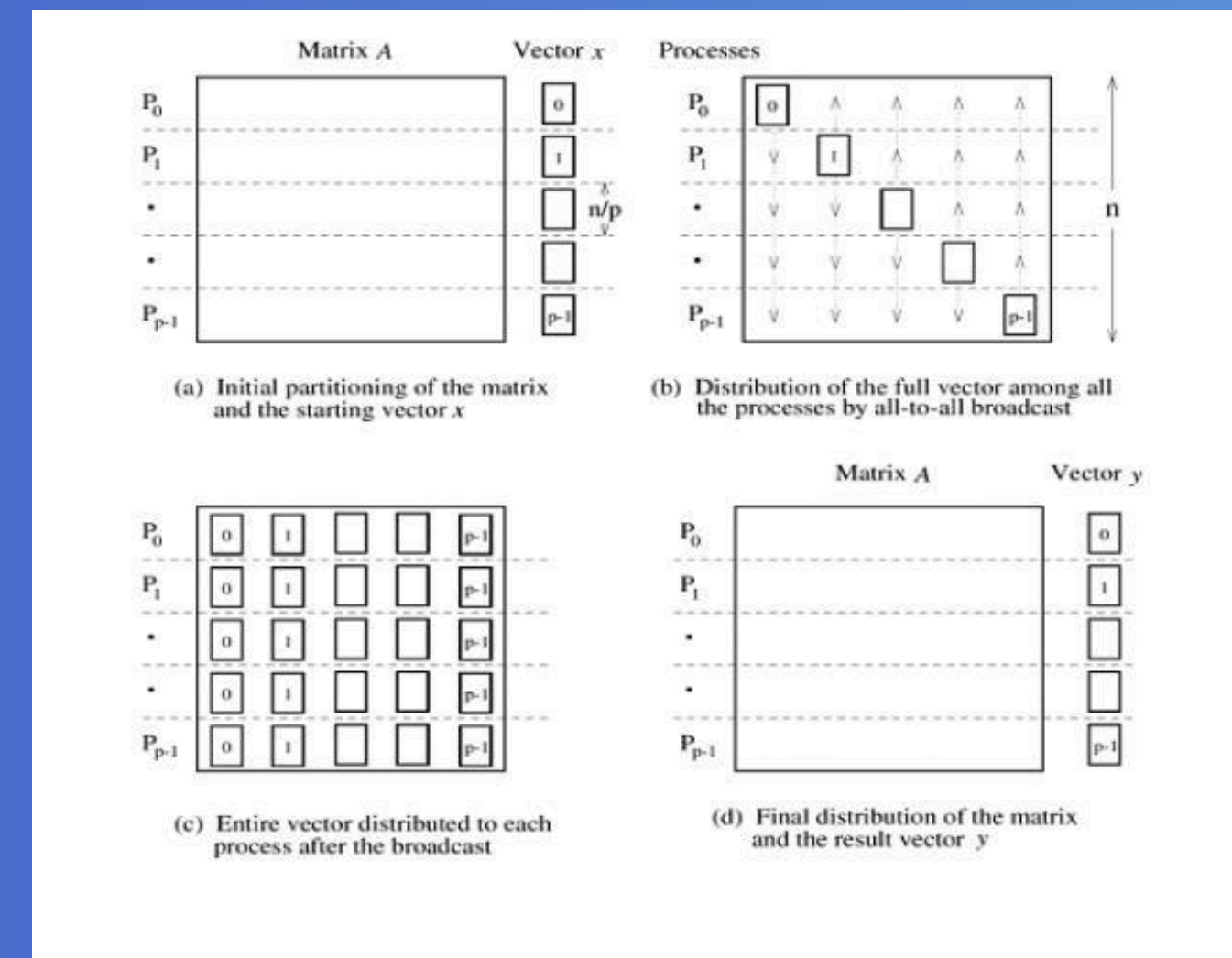
Each processor computes a subset of output vector elements independently.

### Working

Suppose matrix  $A$  has  $n$  rows.

- Processor  $P_1 \rightarrow$  first set of rows
- Processor  $P_2 \rightarrow$  next set of rows
- and so on.

Each processor multiplies its assigned rows with the entire vector.





# MATRIX-VECTOR MULTIPLICATION USING PARTITIONING

## ii) 2D Partitioning

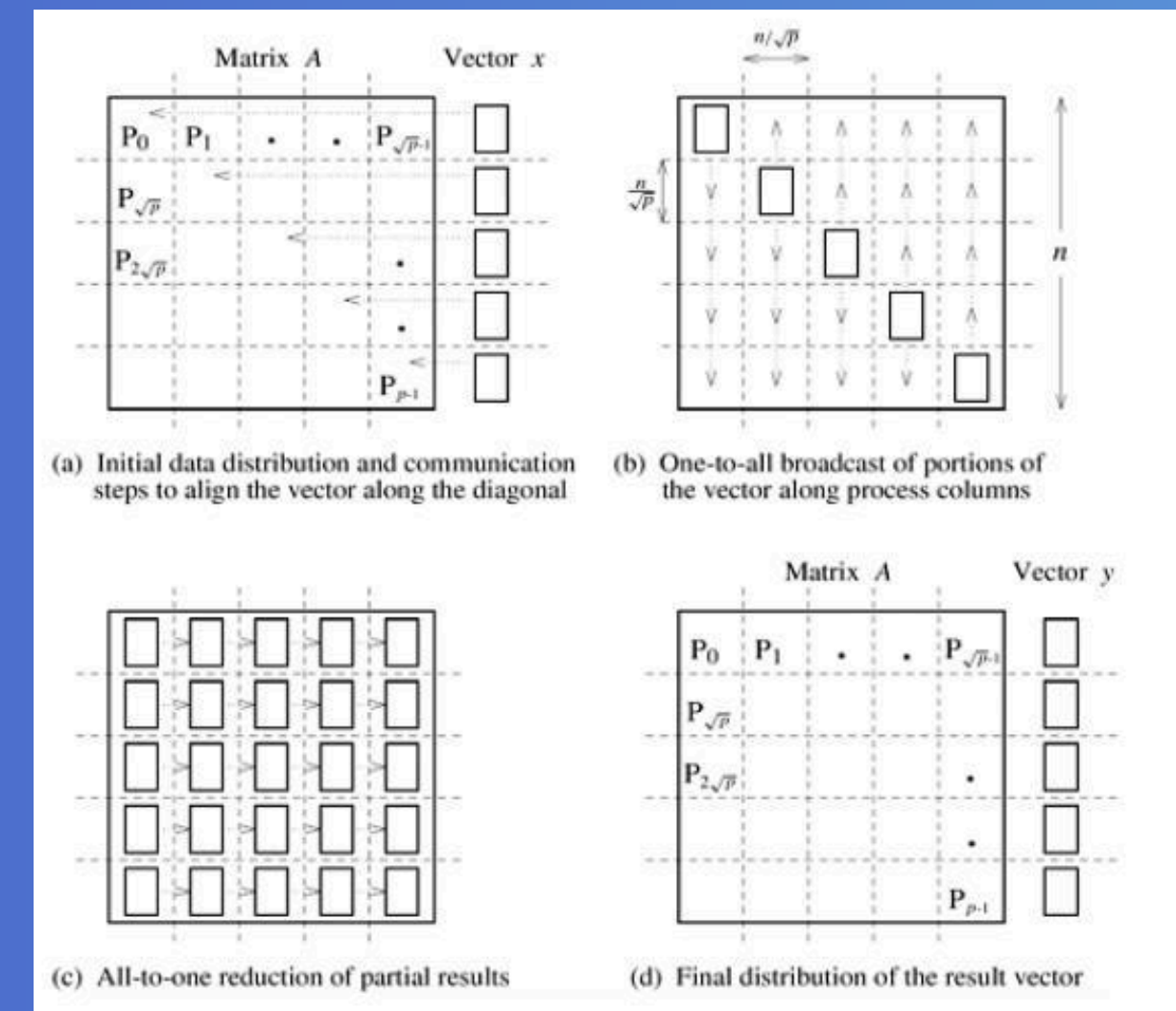
### Definition

In 2D partitioning, the matrix is divided into smaller blocks (submatrices) across rows and columns.

Processors handle different blocks simultaneously.

### Working

- Matrix divided into grid form
- Each processor computes multiplication for its assigned block
- Partial results are combined to form final vector



| 1D Partitioning                   | 2D Partitioning            |
|-----------------------------------|----------------------------|
| Matrix divided row-wise           | Matrix divided into blocks |
| Simple implementation             | Complex implementation     |
| Less communication                | More communication         |
| Limited scalability               | Better scalability         |
| Suitable for small/medium systems | Suitable for large systems |

# MATRIX-MATRIX MULTIPLICATION

- Matrix–Matrix Multiplication is the process of multiplying two matrices to produce a third matrix.
- In parallel computing, different processors compute different parts of the result matrix simultaneously to reduce execution time.

## Matrix Multiplication

Suppose:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Result matrix:

$$C = A \times B$$

## Formula

Each element of matrix  $C$  is calculated as:

$$c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}$$

Where:

- $a_{ik}$  = Elements of matrix A
- $b_{kj}$  = Elements of matrix B
- $c_{ij}$  = Elements of result matrix C



# Parallel Matrix–Matrix Multiplication Algorithm

## Working

- Matrix elements or blocks are distributed among processors.
- Each processor computes some elements of matrix C simultaneously.

## Example

- Processor P1 → computes c11
- Processor P2 → computes c12
- Processor P3 → computes c21
- Processor P4 → computes c22
- All computations occur in parallel.

## Algorithm Steps

1. Divide matrices among processors
2. Each processor multiplies assigned rows and columns
3. Partial results are computed simultaneously
4. Combine all results to form final matrix

**Matrix A (2×2)**

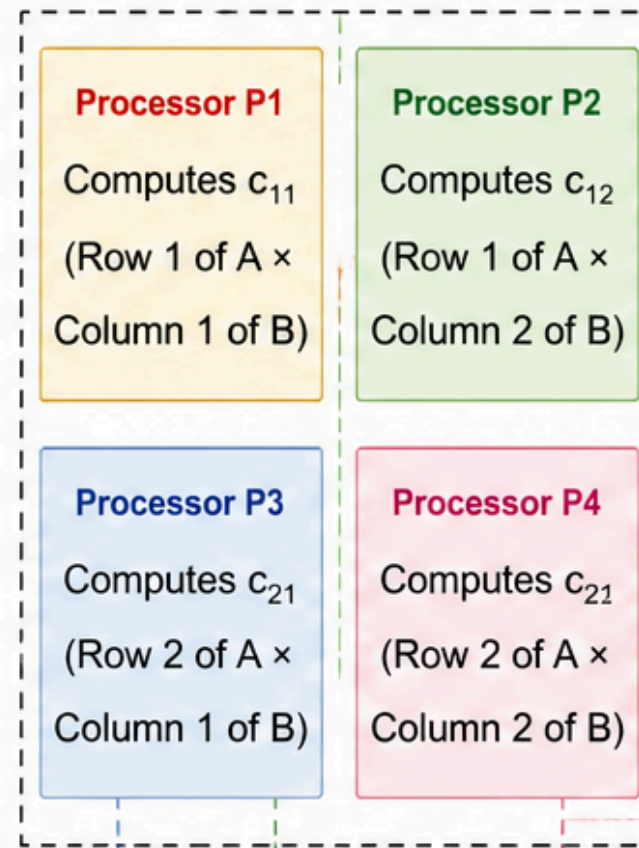
|         | 1st Column    | 2nd Column    |
|---------|---------------|---------------|
| 1st Row | 1<br>$a_{11}$ | 2<br>$a_{12}$ |
| 2nd Row | 3<br>$a_{21}$ | 4<br>$a_{22}$ |

**Matrix B (2×2)**

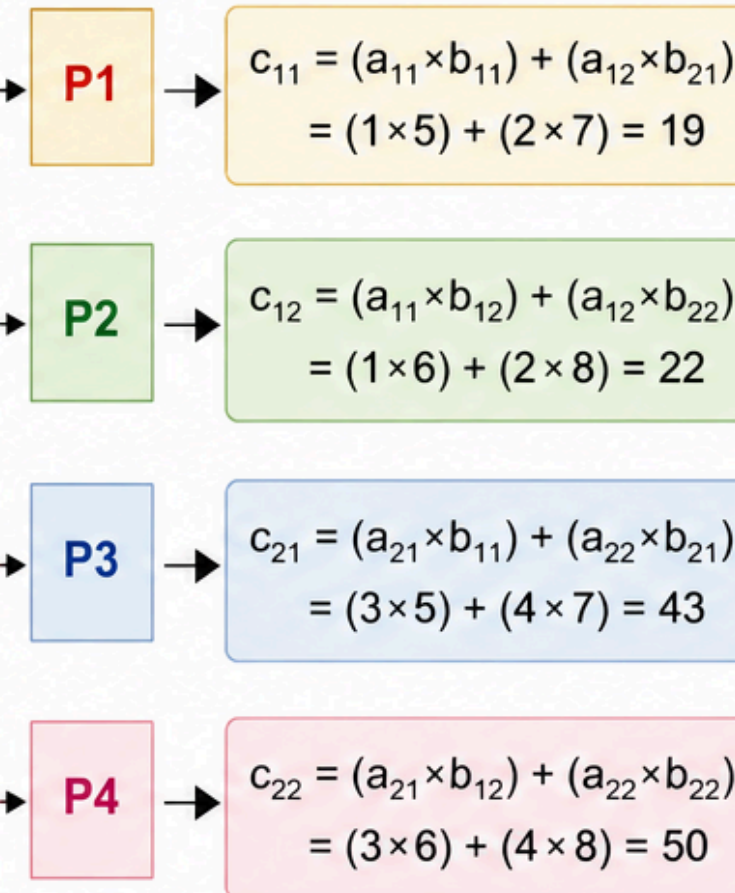
|         | 1st Column    | 2nd Column    |
|---------|---------------|---------------|
| 1st Row | 5<br>$b_{11}$ | 6<br>$b_{12}$ |
| 2nd Row | 7<br>$b_{21}$ | 8<br>$b_{22}$ |

### 1. Distribution of Work

Elements of Result Matrix C are distributed to processors



### 2. Each Processor Computes its Element in Parallel



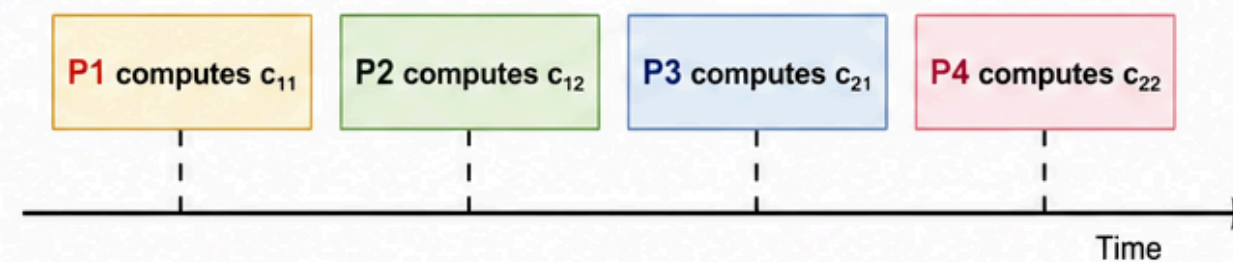
**Result Matrix C = A × B**

|         | 1st Column     | 2nd Column     |
|---------|----------------|----------------|
| 1st Row | 19<br>$c_{11}$ | 22<br>$c_{12}$ |
| 2nd Row | 43<br>$c_{21}$ | 50<br>$c_{22}$ |

#### How it Works?

- The matrices A and B are given.
- The elements of the result matrix C are **divided among processors**.
- Each processor uses the required row of A and column of B to compute its element of C.
- All processors perform computations **at the same time** (in parallel).
- The computed elements are combined to form the final matrix C.

### 3. All Computations Occur in Parallel



**Thus, matrix multiplication is performed faster by distributing the work and computing different parts of the result matrix simultaneously.**

#### Key Points

- Work is divided among processors.
- Each processor computes one or more elements of the result matrix.
- Parallel execution reduces total computation time.
- Communication is minimal in this simple element-wise distribution.

# THANK YOU!



**SUBSCRIBE**

